Loudspeaker project — three-way System D

How we designed it, step by step

34870 Electroacoustics ů Group 10: Louis Andrianne, Sophie Kimura, Mads Rudolph

System D (shared with Group 4): Scan-Speak 26W/8534G00 woofer box + DALI Spektor 2 (S/N 7446912 – L)

What this document is. It is not the hand-in. It is our own explanation of the design: what we simulated, how each simulation works, why we ended up with these values, and how to put the MiniDSP and the filter board together. Tasks 1 and 2 are done in simulation; Tasks 3 and 4 are prepared and wait for lab time. Every number in the text is written by the scripts (`results_task1.tex`, `results_design.tex`), so if the design changes and the scripts are rerun, the text follows. How to rerun everything: `Project/Final design/README.md`.

Contents

1 What we are building **3**

1.1 Words we use a lot . . . 4

2 What we measured, and what each measurement is used for **5**

3 Task 1: tuning the woofer box **5**

3.1 What we can change . . . 5

3.2 How a vented box works . . . 5

3.3 The circuit . . . 6

3.4 Getting the values: fitting the circuit to the impedance . . . 6

3.5 The vent's air mass . . . 7

3.6 Checking the model against the near field . . . 8

3.7 Trying the options and choosing one . . . 8

4 Task 2: designing the crossover **11**

4.1 What a crossover has to do . . . 11

4.2 Getting the real drivers into LTspice . . . 11

4.3 Choosing the two crossover frequencies . . . 12

4.4 The digital filter (woofer and midrange) . . . 12

4.5 The passive filters (midrange and tweeter) . . . 13

4.6 The impedance rule (Lecture 9) . . . 13

4.7 How we found the final values . . . 14

4.8 The result . . . 14

5 Task 3: building it **15**

5.1 Setting up the MiniDSP, step by step . . . 15

5.2 The passive filter board . . . 19

5.3 Measuring the complete system . . . 19

6 Task 4: listening test **20**

7 Conclusion **20**

A Files **20**

1 What we are building

A single loudspeaker driver cannot play the whole audio range well. A big cone moves enough air for bass but becomes directional and starts to break up at high frequencies; a small dome is the other way round. System D therefore has three drivers, and our job is to decide *which driver plays which frequencies*, and to make the three of them add up to one smooth response.

Driver	What it is	What it should play
Woofer	Scan-Speak 26W/8534G00, 26 cm cone, in an 88 L box with vents	the bass, up to about 324 Hz
Midrange	DALI Spektor 2, 5¼ in cone, closed box	about 324 Hz to 3762 Hz
Tweeter	DALI Spektor 2, 25 mm soft dome	everything above about 3762 Hz

The frequency where one driver hands over to the next is a **crossover frequency**. We have two. The project guide fixes *how* each one is made (Figure 1):

- **Woofer ↔ midrange: digital.** The MiniDSP is a small computer that filters the music before the amplifier. Output 1 keeps only the bass and goes through amplifier channel 1 straight to the woofer. Output 2 keeps everything above the bass and goes through amplifier channel 2.
- **Midrange ↔ tweeter: passive.** The signal from amplifier channel 2 goes into a board with coils, capacitors and resistors. A coil lets low frequencies through and blocks high ones, a capacitor does the opposite. That board splits the signal once more between midrange and tweeter.

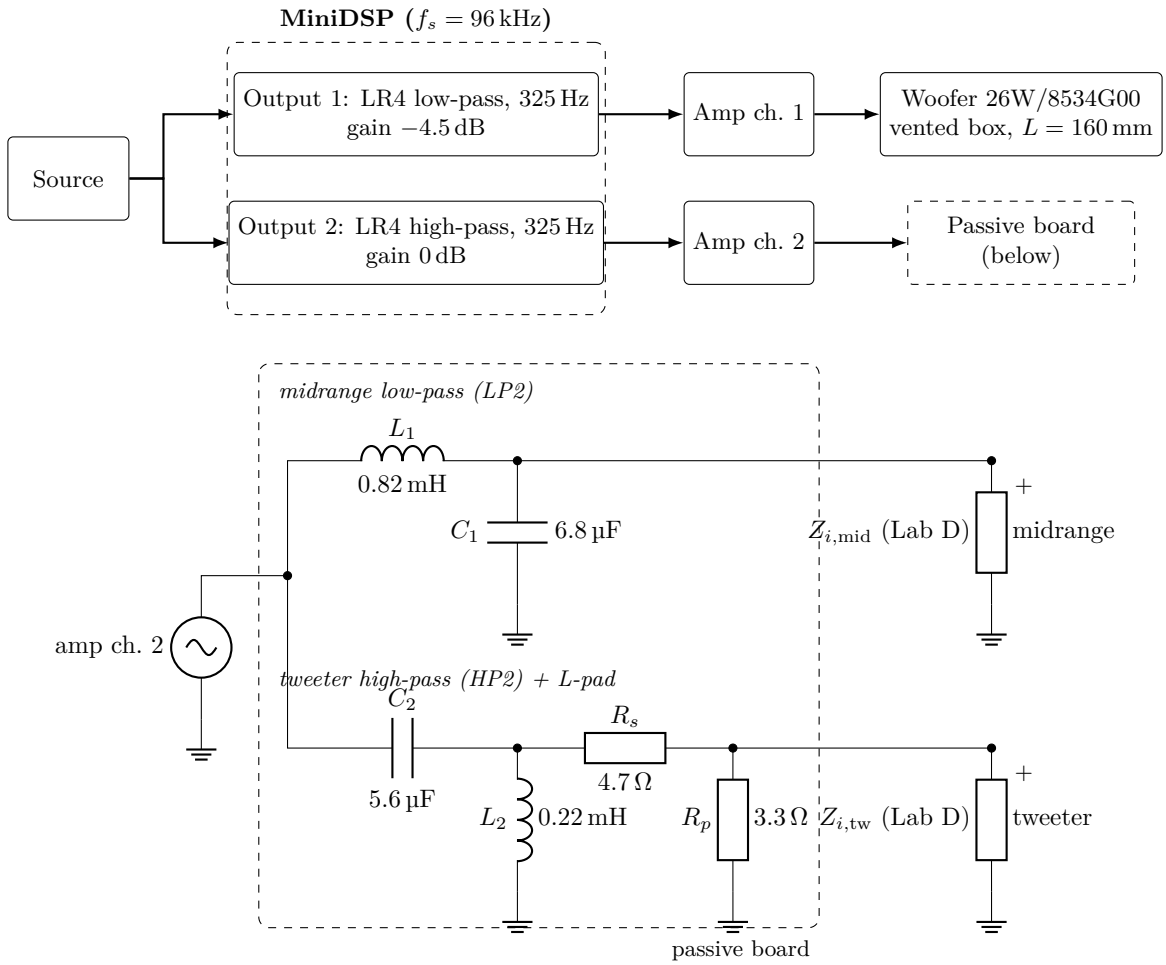


Figure 1: The crossover as it will be built. Top: signal chain. Bottom: the passive board on amplifier channel 2. The drivers are drawn as their measured input impedances Z_i, which is how they load the filters in the simulation; + marks the red terminal (all three units normal polarity). Every part is a single Appendix 1 value. Electrical corners: midrange $1/2\pi\sqrt{L_1C_1} = 2131\,\mathrm{Hz}$, tweeter $1/2\pi\sqrt{L_2C_2} = 4534\,\mathrm{Hz}$; acoustic crossovers 324 Hz and 3762 Hz.

Our final design in one table. The rest of the document explains where each line comes from.

Woofer box	one vent open, length $L = 160\,\mathrm{mm}$, box tuned to $f_B = 24.6\,\mathrm{Hz}$, no extra filling
Digital crossover	LR4 (Linkwitz–Riley, 4th order) at 325 Hz; woofer turned down −4.5 dB
Midrange filter	2nd-order low-pass: coil 0.82 mH in series, capacitor 6.8 µF across the driver
Tweeter filter	2nd-order high-pass: capacitor 5.6 µF in series, coil 0.22 mH across; then an L-pad: 4.7 Ω in series, 3.3 Ω across the tweeter
Wiring	all three drivers normal polarity (red to +)
Result (simulated)	from 150 Hz to 15 kHz within 3.9 dB straight in front, 6.3 dB at 30°

1.1 Words we use a lot

p/V Sound pressure the speaker produces per volt from the amplifier. Our "response". In dB re 1 Pa/V.

Phase How far a signal is shifted in time, as an angle. Two drivers playing the same frequency add up fully when their phases match (0° apart) and cancel when they are 180° apart. This is why every response here has a phase plot.

Z_i The driver's input impedance: what the amplifier "sees" when it pushes current into the driver. It is not a constant resistance; it changes with frequency.

f_B, f_3 Box tuning frequency (where the vent does most of the work) and the frequency where the

	bass has dropped 3 dB, our measure of how deep the speaker plays.
Order	How steep a filter is. 2nd order falls 12 dB per octave, 4th order 24 dB.

2 What we measured, and what each measurement is used for

Everything is built on our own measurements of the actual units, not on data sheets. That is the point of the project: a driver in the lab differs from its data sheet, and the box and the room add their own effects.

Measurement	What it tells us	Where we use it
Lab D: woofer impedance, free air and in the closed box	the driver's mechanical properties (how heavy, how stiff, how strong the motor is)	Task 1: the values in the box circuit
Lab D: woofer impedance with one vent at 160/200/240 mm	how the vent changes the box	Task 1: the vent's air mass
Lab D: near-field pressure at cone and vent	how loud cone and vent really are at low frequencies	Task 1: checking the box circuit
Lab D: midrange and tweeter impedance Z_i	the load the passive filters drive	Task 2: loading the filters correctly
Lab E: p/V of all three units, straight on and at 30°	what each driver sounds like in its box, magnitude and phase	Task 2: the drivers in the crossover simulation

Two limits to keep in mind. The anechoic chamber (Lab E) only works above about 125 Hz, and Lab E was measured with the woofer vents plugged. So the Lab E data says nothing useful about the bass. That is why Task 1 (bass) and Task 2 (everything else) are two separate simulations, which we join at the end.

3 Task 1: tuning the woofer box

3.1 What we can change

The box is already built (88 L), so we cannot pick its size. What we can change is:

- **the vents:** the box has two telescopic tubes. We can plug both (a closed box), open one, or open both, and slide the tube in or out to change its length;
- **filling:** damping wool inside makes the box act slightly larger.

To compare these options we need a model, because the chamber cannot measure the bass and tuning by ear is unreliable.

3.2 How a vented box works

Think of the air in the box as a spring and the air plug inside the vent tube as a small mass sitting on that spring. Like any mass on a spring, it has one frequency where it bounces easily: the box tuning frequency f_B. Around f_B the cone pushes on the box air, the air plug in the vent moves a lot, and *the vent* radiates most of the sound while the cone itself hardly moves. Above f_B the vent's air plug is too heavy to follow and the cone takes over. Below f_B the vent and the cone move in opposite directions and cancel each other, so the bass falls off steeply.

A longer or narrower vent has a heavier air plug, so f_B goes down. Two vents open side by side act like two masses next to each other: the air has two ways out, the total mass is smaller, and f_B goes up.

3.3 The circuit

We draw the woofer and box as an electrical circuit in LTspice (Figure 5). This is the analogy method from Lectures 7 and 8: in every part of the system something pushes and something flows, and we write both as a voltage and a current.

Part	"Voltage"	"Current"	Elements and what they physically are
Electrical	voltage	current	R_E the coil's resistance, L_E the coil's inductance
Mechanical	force	cone velocity	M_{MD} the cone's mass, C_{MS} the suspension's softness, R_{MS} its friction
Acoustical	pressure	air flow (volume velocity)	C_{AB} the box air (the spring), M_{AP} the vent's air plug, R_{AP}, R_{AL} losses in the vent and leaks, and the air in front of the cone (radiation impedance)

The parts are linked by the motor: current in the coil gives a force $F = Bl\,i$, and a moving coil gives a voltage back, $e = Bl\,u$. The cone turns velocity into air flow by its area S_D.

In LTspice the links are controlled sources, as in the lectures: two H sources make $e = Bl\,u$ and $F = Bl\,i$, an E source makes the force $F = S_D\,p$ of the air on the cone, and an F source pushes the cone's air flow $U_D = S_D\,u$ into the acoustic loop. Zero-volt sources (`Vd1` to `Vd5`) work as ammeters, so every flow we need can be read and plotted.

LTspice then solves the circuit at each frequency. Inside the box the cone's air flow U_D splits three ways: into the box air C_{AB} (it gets squeezed), out through the vent (U_P) and through the leak (U_L). Only the part that squeezes the box air is net air leaving the cabinet, and that is what we hear far away. The pressure 1 m away (speaker on the floor, so sound goes into a half space) is

$$p = j\omega\rho\,\frac{U_D - U_P - U_L}{2\pi r} = j\omega\rho\,\frac{U_{AB}}{2\pi r}.$$

The minus signs are there because the vent and leak flows are counted into the box; the vent breathes out when the cone moves in. In the circuit, `Vd5` in series with C_{AB} measures U_{AB} directly, and `Vd4` and `Vd3` give the cone and the vent alone.

We keep two versions of the circuit. `Woofer_Vented_L160.asc` (Figure 5) is the chosen tuning drawn the way the lectures draw it, starting from the circuit our group built for the closed, open and vented boxes; it is the one in the quiz. `Task1_WooferBox.asc` is the same physics with every value as a `.param` and a `.step` over plugged / 160 / 200 / 240 mm; it is the one checked against the fit, and it adds the radiation resistance in front of the cone. The two agree within 0.06 dB in the near field and 0.3 dB in impedance. Figure 6 shows what LTspice gives for the chosen tuning.

3.4 Getting the values: fitting the circuit to the impedance

The circuit is only as good as its values. The data sheet gives a set, but our driver is not exactly the data-sheet driver. So we got the values from our own Lab D impedance curves by *fitting*.

The impedance is a good fingerprint of the driver. Its peak sits at the resonance, and its height and width tell how much damping there is. Fitting means: guess all the unknown values, let the circuit compute its impedance, compare it with the measured curve (magnitude and phase), adjust the values, and repeat until the two curves lie on top of each other. The computer does the adjusting (a standard search method called Nelder–Mead).

We fitted the free-air curve and the closed-box curve at the same time. Free air alone cannot tell the cone's mass from its stiffness. The closed box adds a known spring (the box air, $C_{AB} = V_B/\rho c^2$), which moves the resonance up, and from that shift both are pinned down. $R_E = 5.7\,\Omega$ was measured with a multimeter and S_D was taken from the data sheet.

	Lab D, read off	Lab D, fit	Data sheet*
f_S (Hz), driver resonance	22.7	22.64	22.9
Q_{MS} (mechanical damping)	9.95	9.85	9.1
Q_{ES} (electrical damping)	0.511	0.498	0.41
Q_{TS} (total)	0.486	0.474	0.39
V_{AS} (L), suspension softness as air volume	213	204	165
Bl (T m)	–	8.28	10.1
M_{MS} (g)	–	42.1	50.5
C_{MS} (mm/N)	–	1.174	0.95
R_E (Ω), L_E (mH)	5.7, –	5.7, 1.50	5.7, –

* f_S, Qs and V_{AS} computed from the data sheet's M_{MS}, C_{MS}, Bl, R_{MS} and $S_D = 350\,\mathrm{cm}^2$.

Our driver is softer than the data sheet says (V_{AS} about 25 % larger) and has a weaker motor (Q_{ES} 0.50 instead of 0.41). Both are normal for a used driver from a different batch. If we had used the data sheet, every tuning below would have been off. Figure 2 shows that the fitted circuit reproduces the measured curves.

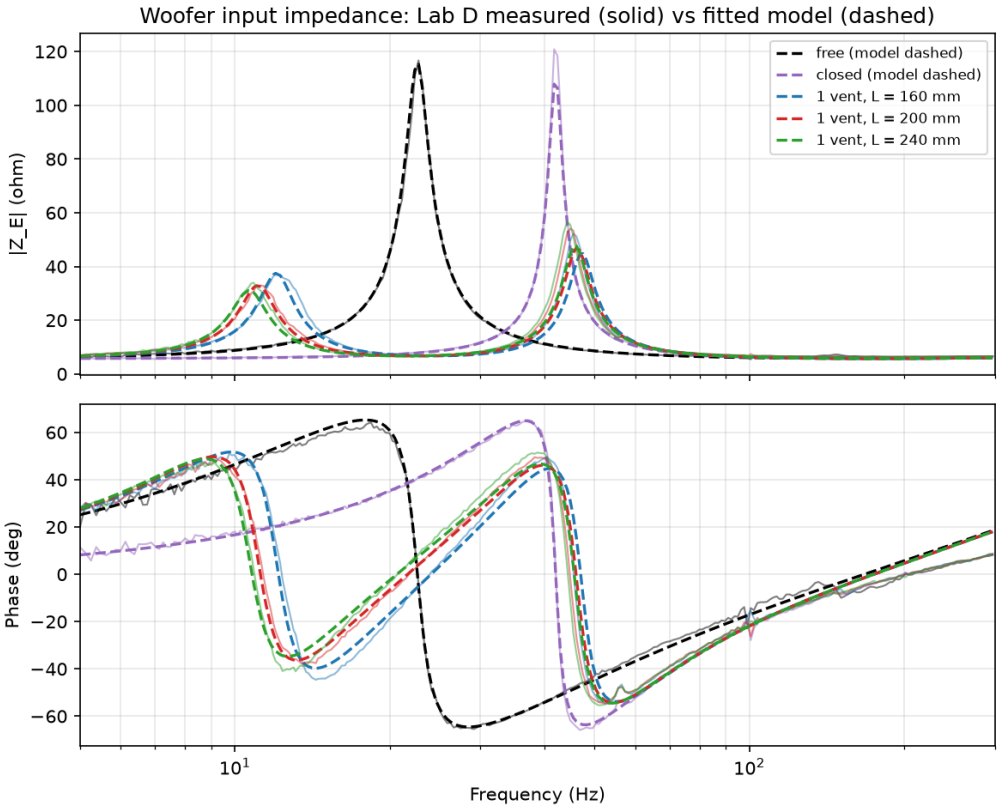


Figure 2: Woofer impedance, Lab D measured (thin lines) and our fitted circuit (dashed), magnitude and phase. Free air (black): one peak at f_S. Closed box (purple): one peak, higher, because the box air stiffens the cone. Vented (coloured): two peaks; the dip between them is f_B, and it moves down as the vent gets longer.

3.5 The vent's air mass

With the driver known, the three vented impedance curves give the vent. The textbook formula for the air plug's mass is $M_{AP} = \rho(L + 1.46a)/\pi a^2$, with L the tube length and a its radius (32.5 mm). The measurement did not follow it: from $L = 160$ to 240 mm, f_B moved only half as much as the formula predicts. So instead of trusting the formula we fitted one mass for each length we measured:

Vent length	M_{AP} fitted (kg/m^4)	M_{AP} formula	f_B (Hz)
160 mm	67.4	75.3	24.6
200 mm	83.0	89.8	22.2
240 mm	91.2	104.3	21.1

The likely reason is that the length we wrote down is how far the telescope was pulled out, not the length the air actually sees, or that the inner tube is narrower than 65 mm. [TODO: measure both vents' inner diameters and the telescope range on the box.]

A second thing the fit showed: with the vents plugged the box has almost no losses (the closed-box peak reaches 121 Ω), but with a vent open the curves need extra losses ($Q_L = 5.2$, $Q_P = 30$). The losses come from the air rushing through the vent, not from leaks in the box. So the model uses different losses for the plugged and the vented box.

3.6 Checking the model against the near field

Fitting the impedance only proves the circuit gets the *electrical* side right. To check that it also gets the *sound* right, we compare it with the Lab D near-field measurements: a microphone almost touching the cone, and one at the vent opening. Very close to a source, the pressure is simply proportional to the air flow through it, $p \approx j\omega\rho U/\pi a$, so the circuit can predict exactly what those microphones should read.

Nothing in Figure 3 is fitted; it is a pure prediction. Cone and vent agree within 2–3 dB, they cross where the measurement crosses, and the cone has its dip right at f_B. That is the cone "resting" while the vent plays, and it confirms the model. Two differences are not the model's fault. Above about 60 Hz the vent microphone also picks up the cone, so the measured vent curve stops falling. Below 120 Hz the room (026) adds ripple from its own resonances.

3.7 Trying the options and choosing one

With a model we trust, we tried every option in LTspice. One run with `.step` goes through the plugged box, the three vent lengths, and a free value for experimenting (Figure 5). Figure 4 shows the result.

Configuration	f_B (Hz)	f_3 (Hz)	ripple (dB)
Vents plugged (closed box)	–	37.4	+0.7
One vent, $L = 160$ mm	24.6	32.9	+1.8
One vent, $L = 200$ mm	22.2	33.4	+1.5
One vent, $L = 240$ mm	21.1	33.8	+1.4
Both vents ($M_{AP} \approx 52$)	28.0	32.7	+2.4

f_3 relative to the average level at 150 Hz to 300 Hz, half space, no room. Ripple = how far the response rises above that level below 150 Hz (a "hump").

How to read it:

- Opening a vent gains about 4 Hz of bass (f_3 37 → 33 Hz) at the cost of a small hump.
- Once a vent is open, *the length hardly matters for* f_3: every f_B from 21 to 28 Hz gives about 33 Hz. Only the hump changes. This is because our driver is very soft compared with the box (V_{AS} is 2.3 times V_B). The standard alignments from Lecture 8 (QB3, B4, C4) assume a specific ratio, so none of them can be reached with this box; the guide says the same.
- The lower plot shows how far the cone moves. Between 25 and 40 Hz the vented cone moves a third as much as the closed one, so it can play louder before it distorts.

Choice: one vent at 160 mm. It gives the same f_3 as the longer settings, cuts the cone excursion, and it is a setting we actually measured, so the near field checks it directly. Opening both vents gives no deeper bass and a bigger hump. Filling would add a few hertz, but we have no measurement with filling to check the model against. A room adds bass of its own, so a hump under 2 dB in this free-space model is on the safe side.

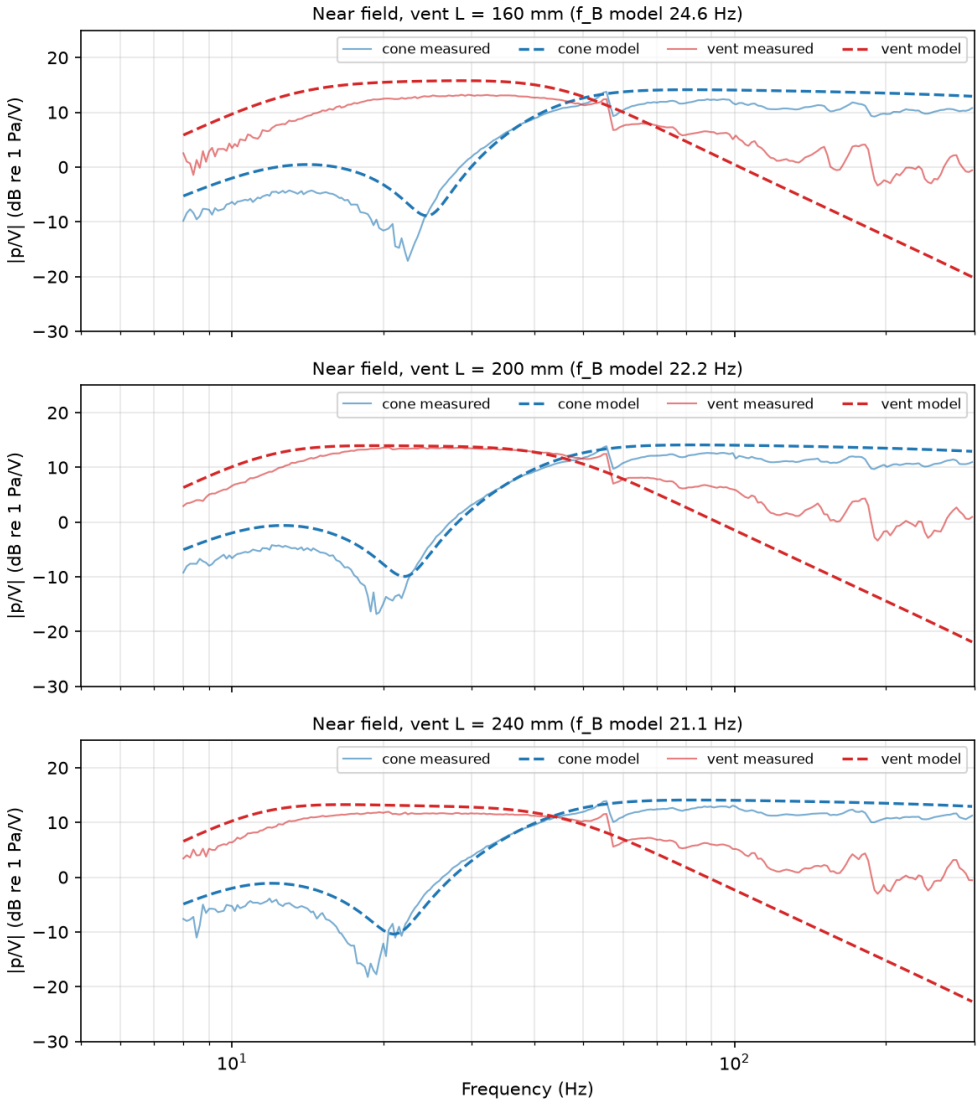


Figure 3: Near-field pressure at the cone (blue) and at the vent (red), measured in Lab D (thin) and predicted by the circuit (dashed), for the three vent lengths. Predicted, not fitted.

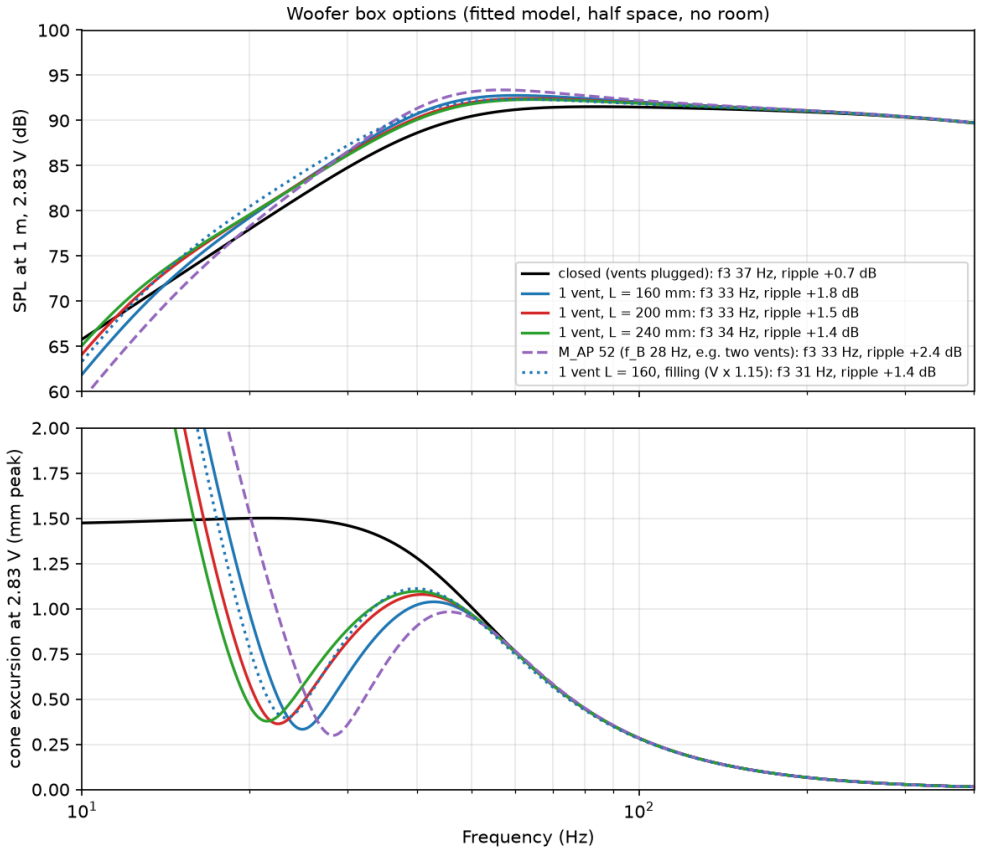


Figure 4: All options at 2.83 V, 1 m, half space. Top: sound pressure level. Bottom: how far the cone moves. Note the vented cone's dip at f_B: the vent does the work there.

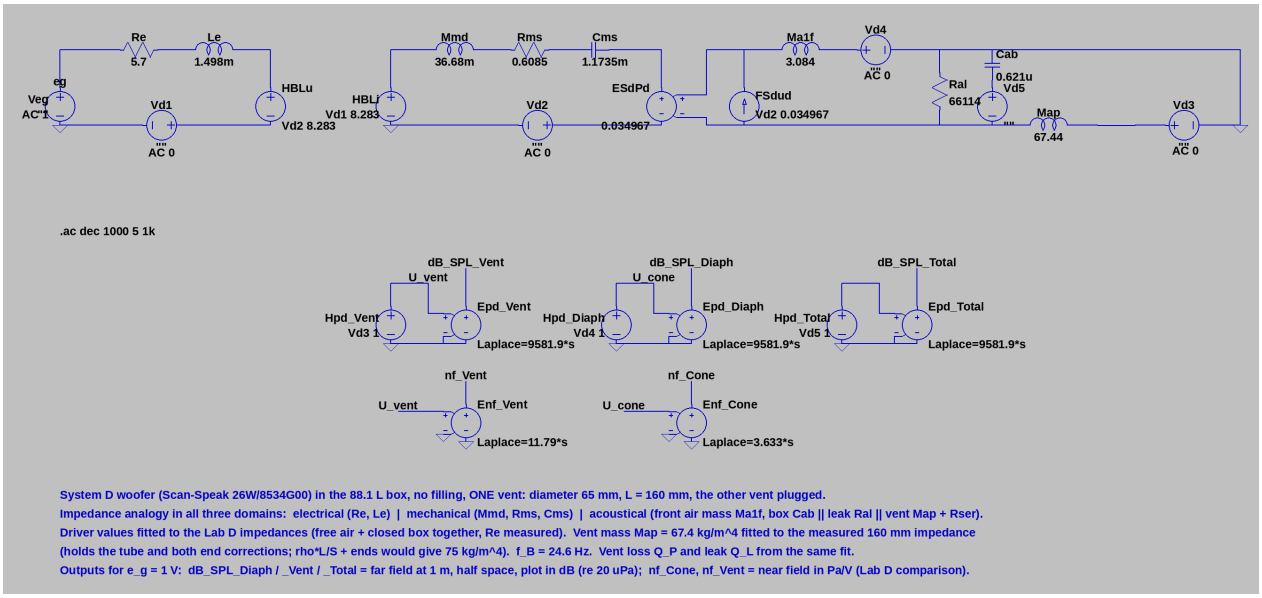


Figure 5: `ltspice/task1/Woofer_Vented_L160.asc`, the chosen tuning (one vent, $L = 160\,\text{mm}$). Top row, left to right: electrical loop (R_E, L_E, back voltage `HBLu` $= Bl\,u$), mechanical loop (force `HBLi` $= Bl\,i$, cone mass M_{MD}, friction R_{MS}, suspension C_{MS}, air force `ESdPd` $= S_D\,p$) and acoustic loop: the cone flow `FSdud` $= S_D\,u$ passes the air mass in front of the cone (`Ma1f`) and splits between the box leak `Ral`, the box air `Cab` and the vent `Map` (its loss $R_{AP} = 348.9$ is the inductor's series resistance). The rows below turn the measured flows into far-field SPL at 1 m and near-field pressure.

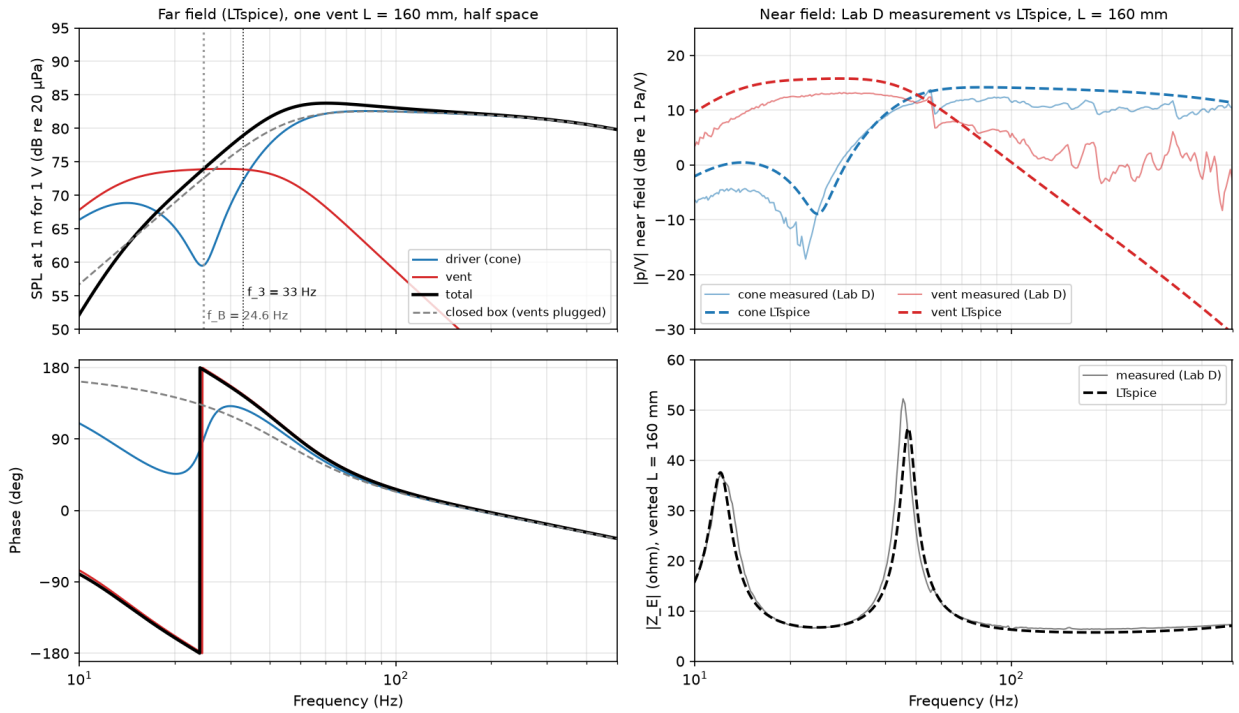


Figure 6: The circuit of Figure 5 as LTspice computes it, for 1 V. Left: far field of the cone, the vent and the total, with the plugged box for comparison: the vent adds bass from 37 down to 33 Hz (f_3) at the cost of a 1.8 dB hump; around f_B the cone nearly stops and the vent does the work. Right: the circuit against the Lab D measurements it was not fitted to (near field), and against the impedance it was fitted to. The near-field level sits about 2 dB high (the simple $p = j\omega\rho U/(\pi a)$ formula and the microphone position); the shapes, the dip at f_B and the impedance peaks match.

4 Task 2: designing the crossover

4.1 What a crossover has to do

Each filter only has to do something simple: let one band through and attenuate the rest. The hard part is the *overlap*. Around each crossover frequency two drivers play at the same time, and the listener hears their *sum*. How they add depends on their phase:

Two equal signals that are	sum compared with one of them	sum compared with the "power sum"
in phase (0° apart)	+6 dB	+3 dB
90° apart	+3 dB	0 dB
180° apart	they cancel	$-\infty$

The phase of each driver's output is the filter's phase *plus* the driver's own phase *plus* the delay from where it sits on the box. So the filter cannot be chosen from a textbook alone. We have to simulate the real drivers. The "power sum" (adding the energies and ignoring phase) is a handy reference: if the real sum lies 3 dB above it at a crossover, the two drivers are in phase there; if it falls below, they are cancelling.

4.2 Getting the real drivers into LTspice

The course script `Matlab2LTspice.m` turns a measured curve into a text file that LTspice can load with `.inc`. Each file defines a special voltage or current source that, at every frequency, follows a table of the measured magnitude and phase. Per driver there are two:

- **the response (TF):** a voltage source that outputs $V(\text{Pout}) = TF(f) \cdot V(\text{Vspkr})$. Put the voltage that reaches the driver in, and the sound pressure the driver would make comes out (as a voltage, 1 V = 1 Pa).

The data is Lab E's p/V at the microphone, 2.44 m away.
- **the impedance (Z_i, midrange and tweeter only):** a current source that draws $I = V/Z_i(f)$ from the filter. To the filter it looks exactly like the real driver connected to it. The woofer does not need one, because nothing sits between the amplifier and the woofer, and an amplifier output keeps its voltage whatever the load.

We checked each unit alone first (`ltspice/units/Unit_*.asc`): LTspice gives back the measured Z_i and p/V to within 0.04 dB.

How an `.inc` file works. Before simulating, LTspice turns the drawing into a text netlist, one line per part. A directive `.inc../data/Zi_mid.txt` means: paste the text of that file into the netlist here. The file holds one part written as text instead of drawn as a symbol, followed by its table, one row per frequency:

```
BiZin_mid Vspkr_mid 0  I=V(Vspkr_mid,0) DB FREQ=
+(2.0000000000E+01,-1.3586246004E+01,-1.2336498083E+01)
+(2.0250000000E+01,-1.3576014100E+01,-1.2341266325E+01)
...                                                    (479 rows)
```

Read left to right: a behavioural current source (`Bi`) named `BiZin_mid`, connected from node `Vspkr_mid` to ground, whose current is the voltage across it multiplied by the table. `DB` says each row is (frequency, dB, phase in degrees); `FREQ=` says the multiplier is the table, interpolated at each frequency of the sweep. The table holds the admittance $Y = 1/Z_i$: at 20 Hz, $|Y| = 10^{-13.59/20} = 0.209$ S, so $|Z_i| = 4.79\,\Omega$. The response file works the same way with a voltage source: `BvTF_midPout_mid0V=V(Vspkr_mid,0)DBFREQ=`. Because these sources have no symbol, the course drawings show dashed boxes where they sit. Three things matter: the path is relative to the `.asc` file; the node names in the first line must match the labels in the schematic exactly (a mismatch gives no error, only a source on an unconnected node); and every unit needs its own source names, so the first line of each file was renamed per unit (`BiZin_mid`, `BiZin_tw`, `BvTF_woof`, ...), as the Project Guide asks. After a run, the SPICE error log lists the files it loaded.

The distance correction. Sound takes 7 ms to travel 2.44 m, and at high frequencies that delay turns the measured phase into a fast spin that is impossible to read. `Matlab2LTspice.m` removes most of it by multiplying with e^{+jkd}, that is, by undoing the delay of a distance d. We used $d = -1.15$ m (chosen in Lab E so the curves read well) and, importantly, *the same d for all three drivers*. The three were measured in one run from one microphone position, so the differences in their phase are real: they come from the drivers sitting at different places on the box. A separate d per driver would erase exactly the information the crossover depends on. The impedances are not corrected at all, because no sound travels anywhere in an impedance measurement.

4.3 Choosing the two crossover frequencies

Before any filter, the drivers themselves tell us roughly where to cross:

- **Woofer to midrange, 300 Hz to 400 Hz.** The woofer cone is big. At about 520 Hz its size equals the wavelength divided by 2π ($ka = 1$), and above that it beams its sound forward; above 1.5 kHz the cone breaks up and the 30° curve falls apart. So it should hand over well before that. The midrange has its resonance at 90 Hz and is flat from about 150 Hz, so it can take over from about 300 Hz.
- **Midrange to tweeter, 2.5 kHz to 4 kHz.** The tweeter has its resonance at 1.45 kHz. A small dome pushed hard at or below its resonance moves a lot and distorts, and its impedance peaks there, which disturbs a passive filter (Section 4.6). So the tweeter crossover must sit well above 1.45 kHz. The midrange is still well behaved at 30° up to about 4 kHz.
- **Levels.** In Lab E the woofer is about 4 dB louder per volt than the midrange, and the tweeter about 8 dB to 10 dB louder. Both have to be brought down to the midrange's level.

4.4 The digital filter (woofer and midrange)

What the MiniDSP does. It samples the music 96 000 times a second ($f_s = 96$ kHz). A *biquad* is the basic filter block: each new output sample is a weighted sum of the newest two input samples, the one before, and the last two output samples,

$$y[n] = b_0\,x[n] + b_1\,x[n-1] + b_2\,x[n-2] + a_1\,y[n-1] + a_2\,y[n-2].$$

Five numbers $(b_0, b_1, b_2, a_1, a_2)$ decide whether it is a low-pass or high-pass, where its corner is and how sharp it bends (its Q). Two biquads in a row give a 4th-order filter.

Why Linkwitz–Riley 4 (LR4). We tried three types (Figure 9):

- **Butterworth 2 (BW2):** the low-pass and the high-pass come out 180° apart, so one driver must be wired inverted, and then they still add to a 3 dB bump at the crossover.
- **Butterworth 4 (BW4):** the course script's $n = 4$. Its outputs are 360° apart (in phase again) but each is only 3 dB down at the crossover, so they add to a 3 dB bump.
- **Linkwitz–Riley 4 (LR4):** each output is 6 dB down at the crossover and the two are in phase at *every* frequency. Two halves in phase add back to exactly the original: a flat sum, no bump, normal polarity. This won.

An LR4 is two identical Butterworth 2nd-order biquads in a row ($Q = 0.707$ each). The course script `DigitalCrossover.m` only makes Butterworth filters, so we ran it with $n = 2$ (one BW2 biquad) and used the same biquad twice. Our own script does the same thing (the `bq` function in `design/xover.py` is a copy of the script's `BQ`), and the numbers are identical.

The digital filter in LTspice. LTspice cannot run a sampled filter, but it can use e^{-sT}, which is a delay of exactly one sample ($T = 1/f_s = 10.4\,\mu s$). Writing the biquad with e^{-sT} in place of each "previous sample" gives a `Laplace=` string for an E source that behaves exactly like the MiniDSP. Two E sources in a row make the LR4.

The woofer level. Output 1's gain is −4.5 dB, which matches the woofer's extra sensitivity. It costs nothing in a digital crossover, because each output has its own amplifier channel. (Turning the woofer down with resistors would spoil the box tuning, Lecture 9.)

4.5 The passive filters (midrange and tweeter)

Starting point: the textbook. A 2nd-order low-pass is a coil in series and a capacitor across the driver: at high frequencies the coil blocks and the capacitor shorts what gets through. A 2nd-order high-pass is the mirror image, a capacitor in series and a coil across. The Lecture 9 formulas, assuming the driver is a plain resistor R_E, give for a Butterworth at 3762 Hz:

	textbook (driver = resistor)	what we chose
midrange low-pass, $R_E \approx 5\,\Omega$	$L = 0.30$ mH, $C = 6.0\,\mu$F	0.82 mH, 6.8 µF
tweeter high-pass, $R_E \approx 3.6\,\Omega$	$C = 8.3\,\mu$F, $L = 0.22$ mH	5.6 µF, 0.22 mH, + L-pad

The midrange values differ a lot from the textbook. That is not a mistake; the next section explains why.

4.6 The impedance rule (Lecture 9)

A passive filter is a voltage divider between its parts and the driver, so what comes out depends on the driver's impedance. The textbook assumes a constant resistor. A real driver is not one:

- **Midrange:** the voice coil is a coil, so the impedance rises with frequency: 5 Ω at 300 Hz, 8.5 Ω at 2 kHz, 11.7 Ω at 3.8 kHz, 13.6 Ω at 5 kHz. A filter calculated for 5 Ω sees more than twice that at the crossover, so it rolls off later and with a different shape than planned.
- **Tweeter:** its impedance has a peak at the resonance, 6.7 Ω at 1.45 kHz against about 4 Ω elsewhere. A high-pass near that peak gives a bump in the output.

Lecture 9's advice, and what we did about each point:

1. **Always simulate the filter with the real driver as its load.** Our filters drive the measured Z_i in every simulation. That is why the midrange values "look wrong": the search chose a larger coil (corner $1/2\pi\sqrt{LC} = 2131$ Hz) because the midrange's rising impedance makes the filter rise slightly before it rolls off. That rise fills in the midrange's own natural roll-off, and the acoustic result crosses at 3762 Hz (Figure 7, orange).
2. **Keep the high-pass away from the driver's resonance.** The tweeter's electrical corner is 4534 Hz and the acoustic crossover 3762 Hz, both well above 1.45 kHz. At the resonance the tweeter is 17.5 dB below the rest of the system.
3. **No Zobel networks** ("you are not allowed to use it in your project"). A Zobel would flatten the impedance, but it is sensitive to its values and needs non-standard parts. We moved the corner frequencies instead, as the lecture suggests.

4. **L-pad only in the passive branch.** See below.

The L-pad. The tweeter is about 10 dB too loud. An L-pad is two resistors: $R_s = 4.7\,\Omega$ in series and $R_p = 3.3\,\Omega$ across the tweeter. With the tweeter at about $4.2\,\Omega$, $R_p \parallel Z_{tw} \approx 1.8\,\Omega$, and the divider gives $1.8/(4.7 + 1.8) = 0.28$, which is -11.1 dB. A useful side effect: the filter now sees about $6.5\,\Omega$ of mostly resistor instead of the tweeter's bumpy impedance, so the tweeter's resonance peak disturbs it much less.

4.7 How we found the final values

There are many combinations: crossover frequency, filter type, woofer gain, every standard coil, capacitor and resistor, and the three polarities. Trying them by hand in LTspice would take days. So we wrote the whole system in Python (`design/xover.py`), doing the same calculation as the LTspice circuit, and let it search.

What the model computes. For 1 V at the MiniDSP input, each driver's sound is the chain of everything in front of it, multiplied together:

$$\begin{aligned} \text{woofer} &= (\text{digital low-pass}) \times (\text{gain}) \times TF_{\text{woofer}} \times (\text{Task 1 box}) \\ \text{midrange} &= (\text{digital high-pass}) \times (\text{passive low-pass loaded by } Z_{i,\text{mid}}) \times TF_{\text{mid}} \\ \text{tweeter} &= (\text{digital high-pass}) \times (\text{passive high-pass + L-pad loaded by } Z_{i,\text{tw}}) \times TF_{\text{tw}} \end{aligned}$$

and the system is the sum of the three, *with phase* (complex numbers). The "Task 1 box" term swaps the plugged woofer Lab E measured for our vented one: it is the ratio of the vented to the plugged response from the Task 1 model, so it only changes the bass. The passive filters are solved like a chain of voltage dividers, part by part from the amplifier to the driver.

What the search does. It starts from a reasonable design and changes one thing at a time (the crossover frequency in steps of 25 Hz, the filter type, the woofer gain in 0.5 dB steps, each part to the next standard value, each polarity). It keeps the change whenever the score gets better, and stops when nothing helps. The score is lower (better) when:

- the summed response from 150 Hz to 15 kHz is flat, straight on and (counted half) at 30°, after smoothing over 1/6 octave so single measurement wiggles don't count;
- the sum never falls below the power sum (no cancelling);
- the crossovers stay in the ranges from the previous section, and the tweeter is at least 12 dB down at its resonance.

The last rule matters. Without it, the search pulled the tweeter crossover down to 2 kHz: flat on paper, but it would make the tweeter work near its resonance, where it distorts.

Checking with LTspice. The final design was then drawn in LTspice the way the course examples are drawn (`ltspice/System_D_course.asc`, Figure 11): the two digital E sources per output, the passive parts, the measured driver files, and the guide's summing E sources with a polarity factor per driver. LTspice and the Python model agree to 0.00 dB at every measured frequency, so the Python search and the course tool give the same answer (Figure 12).

4.8 The result

The voltage at each driver (Figure 7, top) shows what the filters do before the drivers' own behaviour is added. The tweeter line sits about 10 dB lower because of the L-pad. The bottom plot is what amplifier channel 2 has to drive. It never goes below $4.4\,\Omega$, which is safe for the amplifier.

The sound (Figure 8): straight on, the three drivers add up to a response that stays within 3.9 dB from 150 Hz to 15 kHz (typical deviation 0.90 dB). At 30° it is 6.3 dB, mainly because the tweeter and the woofer get more directional at their top ends.

The phase (Figure 9): at both crossovers the drivers' phase curves lie on top of each other, and the sum is exactly 3 dB above the power sum there: fully in phase, and the sum never drops below the power sum anywhere. With 2nd-order passive filters one would normally expect to have to invert the tweeter. Here the drivers' own roll-offs and their positions on the box add enough phase that normal polarity is the one that lines up.

Tolerances (Figure 10): real parts are not exact. We simulated 400 boards with random part errors (coils ±5 %, capacitors ±10 %, resistors ±5 %). 95 % of them stay within a band at most 1.7 dB wide, and the coils'

own resistance (0.3–0.6 Ω per mH) changes the response by at most 0.45 dB. The design does not depend on lucky parts.

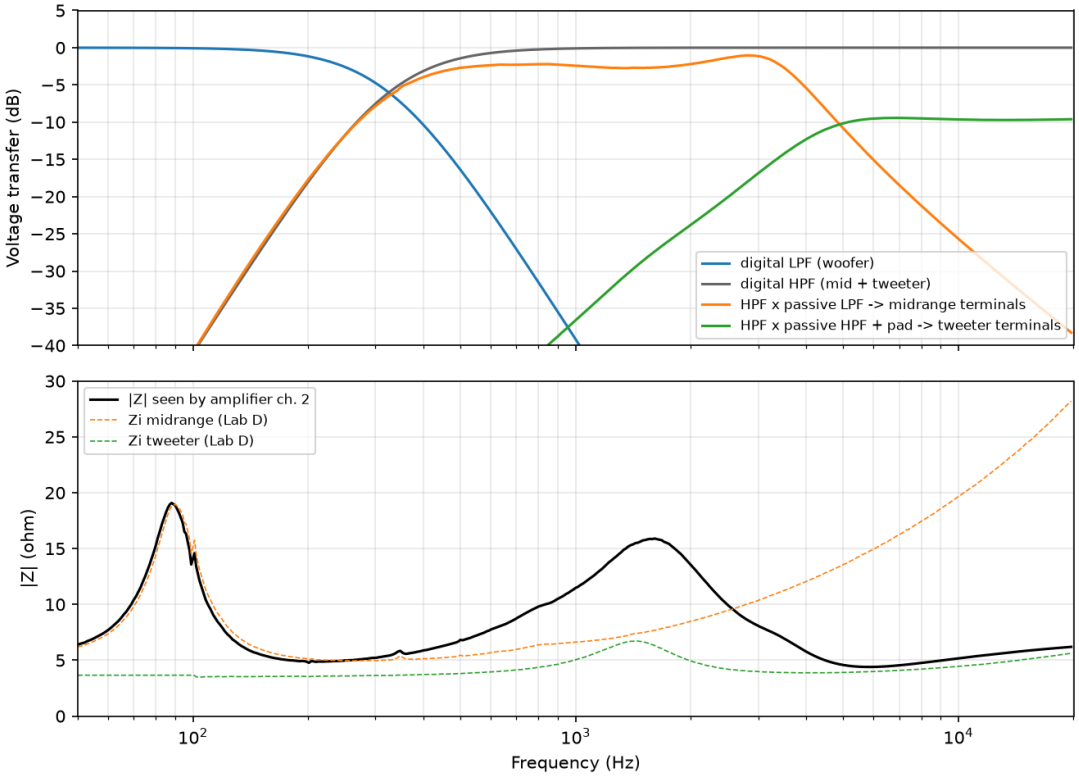


Figure 7: Top: voltage at each driver's terminals per volt in, from the digital filters (blue, grey) and after the passive filters loaded by the real drivers (orange, green). Bottom: impedance seen by amplifier channel 2 (black) and the two drivers' measured Z_i (dashed).

5 Task 3: building it

5.1 Setting up the MiniDSP, step by step

The MiniDSP only needs the two LR4 filters and the woofer gain. Everything else must be switched off, or it would add filtering we did not simulate.

1. Connect the MiniDSP by USB and open the MiniDSP plugin on the lab PC.
2. **Routing:** in the input/output matrix, turn on only *Input 1* → *Output 1* and *Input 1* → *Output 2*. All other routes off.
3. **Bypass everything except the crossover** on both outputs: PEQ, compressor and FIR set to bypass, only *Xover* active. Leave the polarity/invert buttons off; our polarities are all normal.
4. **Output 1 (woofer):** Outputs → Output 1 → Xover → *Advanced*. Paste the text below (it is in `data/dsp/MiniDSP_woofer_out1.txt`) and press *Process*. These are two identical low-pass biquads at 325 Hz, which together are the LR4.

```
biquad1,
b0 = 0.000111435745339,
b1 = 0.000222871490678,
b2 = 0.000111435745339,
a1 = 1.969920198141118,
a2 = -0.970365941122475,
biquad2,
b0 = 0.000111435745339,
b1 = 0.000222871490678,
b2 = 0.000111435745339,
a1 = 1.969920198141118,
a2 = -0.970365941122475,
```

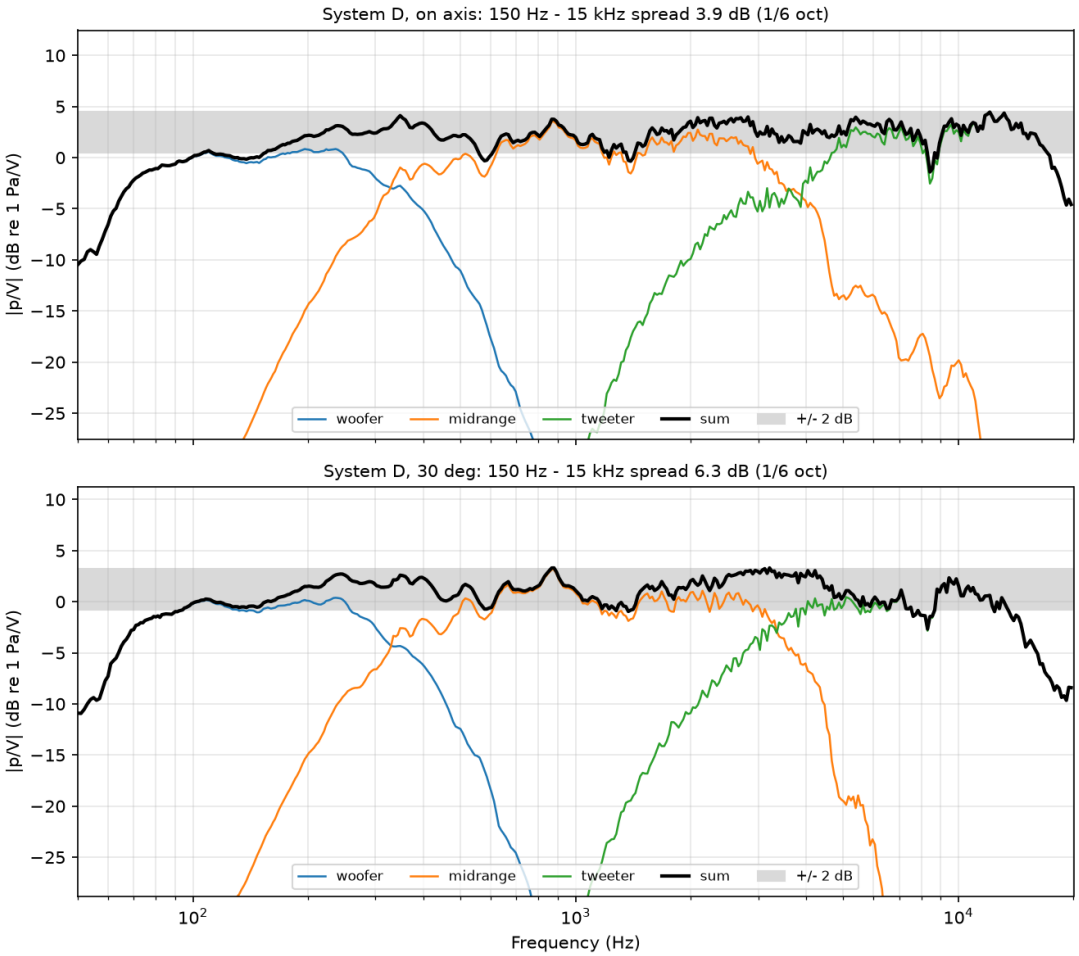


Figure 8: Simulated sound from each driver and the sum, straight on (top) and at 30° (bottom). The grey band is ±2 dB around the average level.

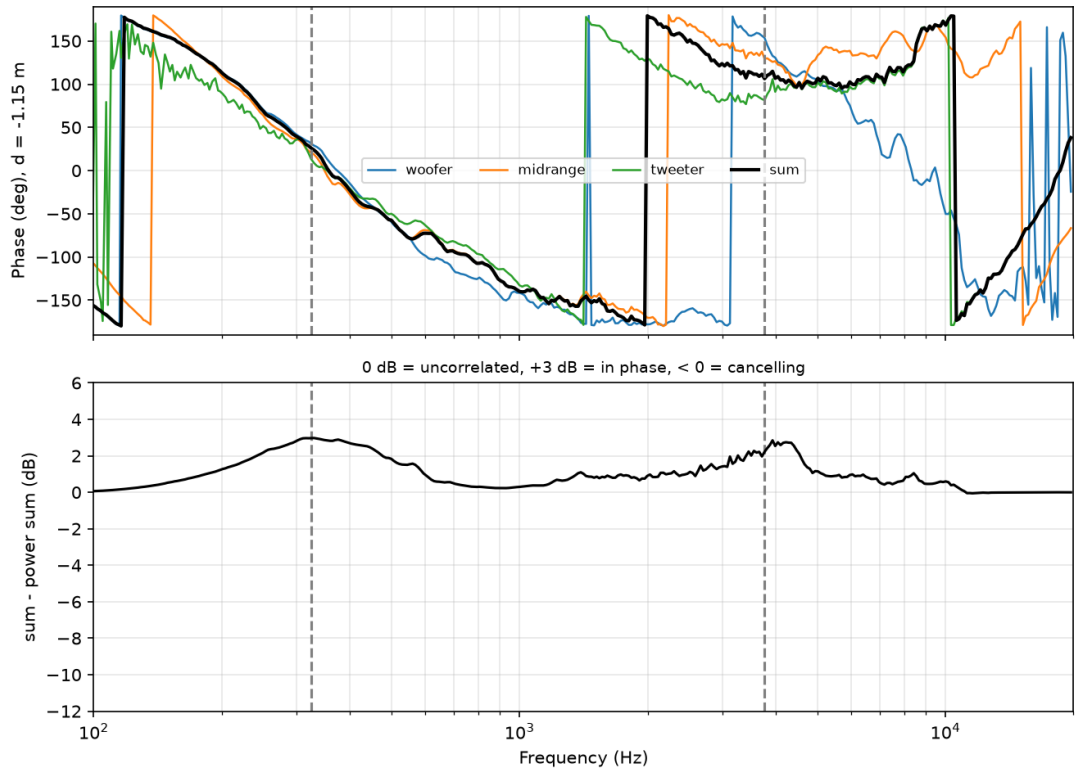


Figure 9: Top: phase of each driver's sound (same distance correction $d = -1.15\,\mathrm{m}$ for all) and of the sum. Where two curves overlap at a crossover (dashed lines), the drivers are in phase. Bottom: the sum compared with the power sum; +3 dB at a crossover = in phase, below 0 = cancelling.

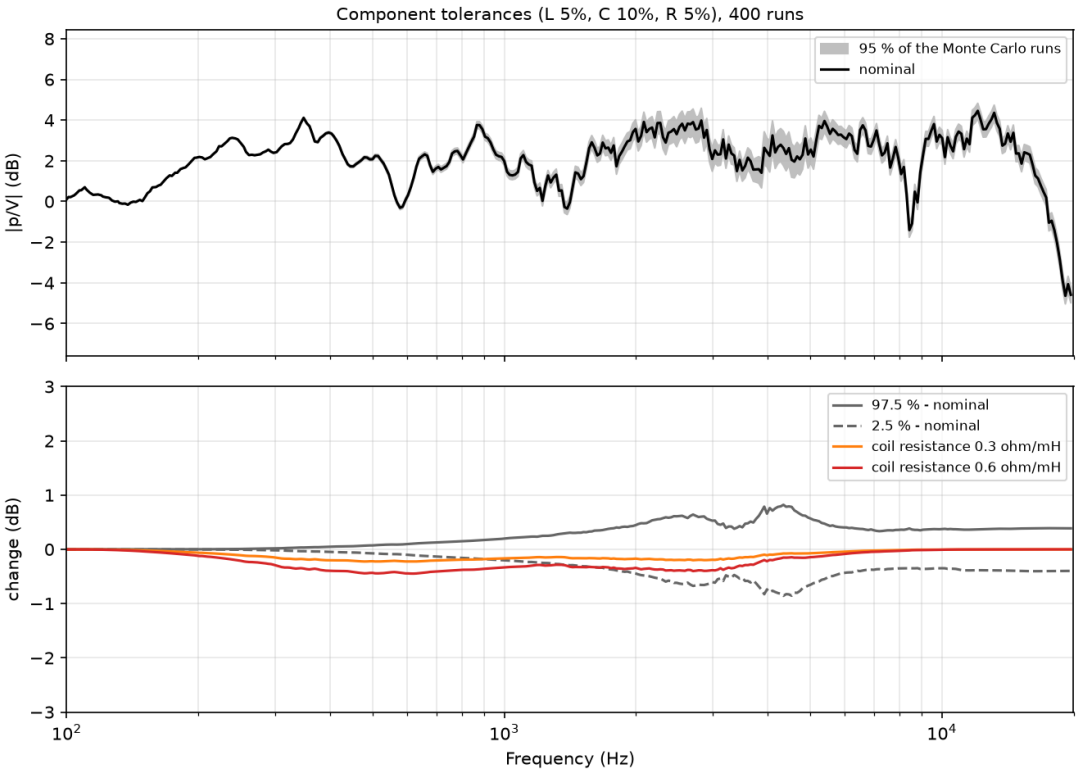


Figure 10: Top: 400 simulated boards with random part tolerances; the grey band holds 95 % of them. Bottom: the same as a difference from the nominal design, plus the effect of the coils' resistance.

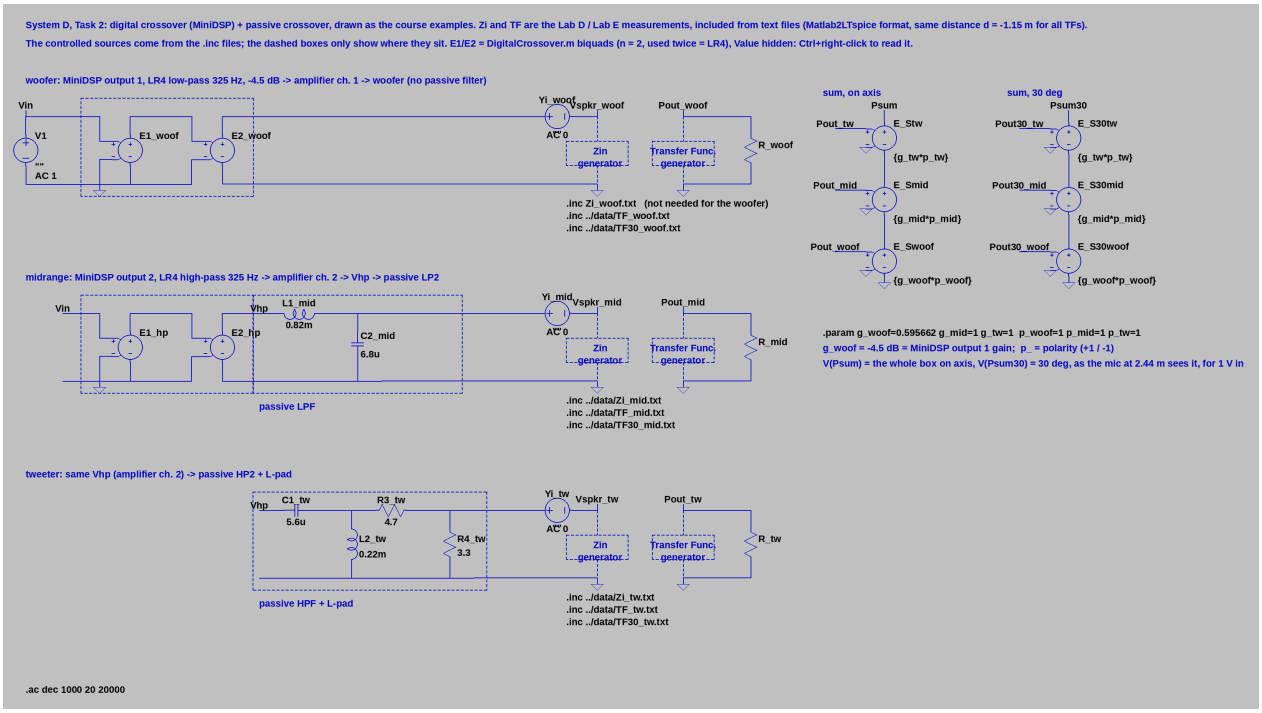


Figure 11: `ltspice/System_D_course.asc`, the complete system drawn like the course examples: one row per driver, each the course template. Signal `Vin` → digital biquads `E1`, `E2` (the `Laplace=` strings are hidden) → passive filter → ammeter `Yi` → `Vspkr` → the measured impedance (Zin) and response (Transfer Func.) from the `.inc` files → `Pout`. Woofer: digital low-pass only, its Z_i left out on purpose. Midrange and tweeter share amplifier channel 2 (`Vhp`): LP2, and HP2 with L-pad. Right: the series E sources add the three outputs with the woofer gain and the polarities; `V(Psum)` is the system on axis, `V(Psum30)` at 30°.

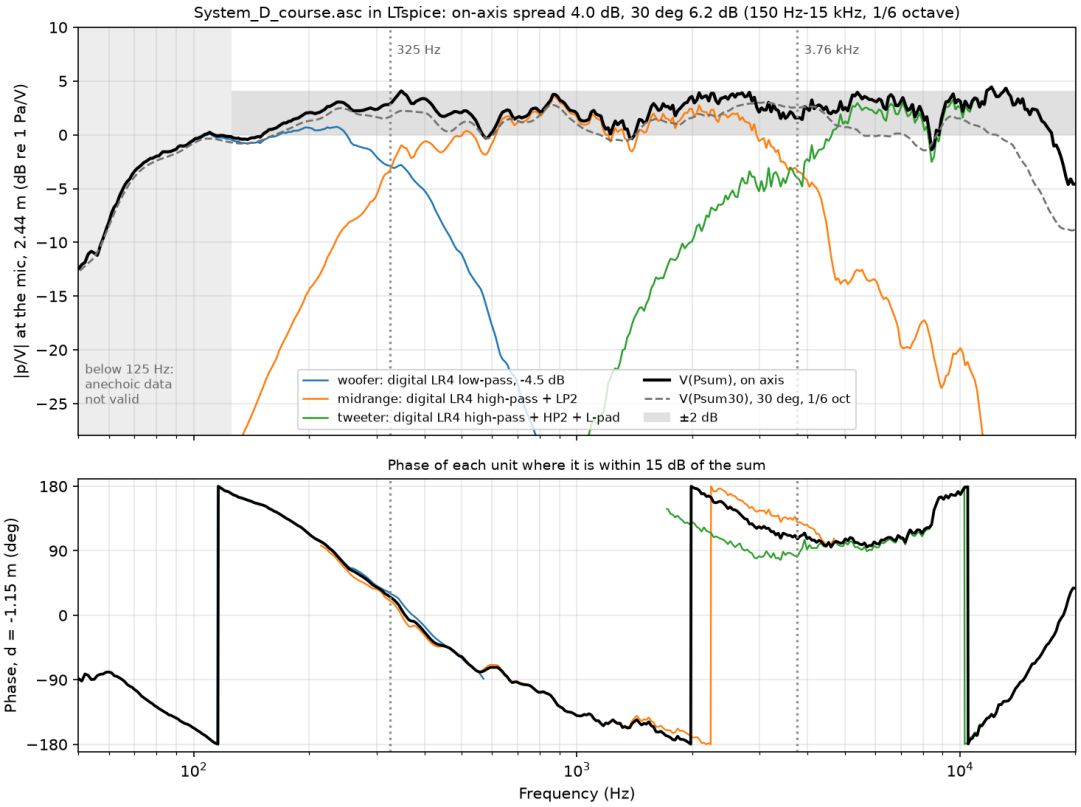


Figure 12: Figure 11 as LTspice computes it: each driver's sound at the microphone (woofer including its −4.5 dB) and the sum, with the phase below. The result is the same as Figure 8 from the Python model (they agree to 0.00 dB). Below 125 Hz the anechoic data does not hold; the low end is Task 1's job.

5. **Output 2 (midrange + tweeter):** the same for Output 2 with the high-pass text (`data/dsp/MiniDSP_midtw_out2.txt`):

```
biquad1,
b0 = 0.985071534815898,
b1 = -1.970143069631796,
b2 = 0.985071534815898,
a1 = 1.969920198141118,
a2 = -0.970365941122475,
biquad2,
b0 = 0.985071534815898,
b1 = -1.970143069631796,
b2 = 0.985071534815898,
a1 = 1.969920198141118,
a2 = -0.970365941122475,
```

6. **Gains:** Output 1 −4.5 dB, Output 2 0 dB. Input gain so the amplifier does not clip.
7. Save the configuration on the MiniDSP and in a file, so the demonstration on 3 Dec starts from the same settings.

Reading the numbers. The MiniDSP's `a1`, `a2` carry the opposite sign of the textbook form (it adds the old outputs, as in the equation in Section 4.4). That is why `a1 = 1.9699` here, while the LTspice string in `data/dsp/LTspice_*.txt` has −1.9699. Both are the same filter. The numbers only work at 96 kHz: at another sample rate the same coefficients would put the crossover somewhere else. If the design changes, rerun `python3 xover.py -export` (or `DigitalCrossover(f0,'LPF',2)` in MATLAB, pasted twice) and paste the new text.

Quick check after setting it up: play a sine at 325 Hz and measure Output 1 and Output 2 with the oscilloscope. Both should be about 6 dB below the input (Output 1 a further 4.5 dB lower), and the two should be in phase.

5.2 The passive filter board

Figure 1 is the circuit to build. All values are single parts from the course basket (Appendix 1 of the guide); we are not allowed to combine parts to get in-between values.

Branch	Part	Value	Where it goes
Midrange	coil	0.82 mH	in series, from amplifier + to the midrange +
Midrange	capacitor	6.8 µF	across the midrange (+ to −)
Tweeter	capacitor	5.6 µF	in series, from amplifier +
Tweeter	coil	0.22 mH	from after the capacitor to ground
Tweeter	resistor R_s	4.7 Ω	in series, after that node
Tweeter	resistor R_p	3.3 Ω	across the tweeter (+ to −)

Both branches start at amplifier channel 2. Wire both drivers red to +. Keep the two coils apart and at right angles to each other, so one does not pick up the other's field. [TODO: photo of the board, actual parts used, measured values if an LCR meter is available.]

5.3 Measuring the complete system

Plan for the anechoic chamber (vents covered; the chamber cannot measure below 125 Hz anyway):

1. Each driver alone through its whole chain (MiniDSP active, the other output muted). Compare with the single-driver curves in Figure 8. If one is off, the fault is in that branch.
2. All three together, 0° and 30°. Compare with the sum.
3. Flip the woofer's wires: there must be a deep dip at 324 Hz. Flip the tweeter: a dip at 3762 Hz. A deep dip proves the drivers really are in phase there with normal wiring, so this checks both the wiring and our phase model.

[TODO: measured vs. simulated, and what we changed.]

6 Task 4: listening test

In the IEC 60268-13 listening room (Lecture 10), planned:

- **Set-up:** the box at the standard position, the listener 2.5 m away straight in front, ears at about tweeter height. Level set to 80 dB(A) on pink noise with a sound level meter, so "louder" does not win by default.
- **Comparison:** A/B pairs. A = our design, B = one change (woofer gain ±2 dB, tweeter R_s one value up or down, vent plugged vs. open). The operator switches; the listener does not know which is which. Every pair is played twice, in both orders.
- **Music:** 20–30 s clips with a male voice, acoustic guitar or piano, steady bass, and pink noise.
- **Questions:** per clip, which one they prefer (A, B or no difference), and 1–5 for amount of bass, natural voices, brightness, and overall. Free comments.
- **Listeners:** at least 6, including people outside the group, after a short practice round.

[TODO: results (count preferences, average scores per question) and what we changed because of them.]

7 Conclusion

[TODO: after Tasks 3 and 4: final design, measured vs. simulated, what we would change.]

A Files

Everything lives in `Project/Final design/` (the export of the raw lab data is in `Project/matlab/export_systemD.m`).

File	What it is
`design/design.json`	the design: every value in this document
`design/xover.py`	the whole system in Python; `--plots` makes the Task 2 figures, `--export` the MiniDSP text
`ltspice/task1/box_model.py`	the Task 1 fit and the tuning table; `--plots` makes the Task 1 figures
`ltspice/task1/Woofer_Vented_L160.asc`	the chosen woofer box, lecture style (made by `gen_vented_course.py`; `plot_vented_ltspice.py` runs it and draws Figure 6)
`ltspice/task1/Task1_WooferBox.asc`	the woofer box with all tunings as a `.step` (made by `gen_box_ltspice.py`)
`ltspice/System_D_course.asc`	the complete system, course style (made by `gen_system_course.py`; `plot_system_ltspice.py` runs it and draws Figure 12)
`ltspice/System_D_design.asc`	the same system with the Task 1 box added to the woofer, used for the design search (made by `gen_system.py`)
`ltspice/units/`	each driver alone with its measured data, no filter
`data/`	the LTspice data files, `csv/` the same data for the scripts, `dsp/` the filter text
`app/index.html`	Crossover Studio: change any value and see the response, in the browser

The `.asc` files are generated: change `design.json` or the scripts and rerun them rather than editing the schematics by hand. `--verify` on either generator runs LTspice and compares it with the Python model.